



Exploring and pre-processing a dataset using Pandas

Estimated time needed: **30** minutes

Objectives

After completing this lab you will be able to:

- Explore the dataset
- Pre-process dataset as required (may be for visualization)

Introduction

The aim of this lab is to provide you a refresher on the **Pandas** library, so that you can pre-process and analyse the datasets before applying data visualization techniques on it. This lab will work as a crash course on *pandas*. If you are interested in learning more about the *pandas* library, detailed description and explanation of how to use it and how to clean, munge, and process data stored in a *pandas* dataframe are provided in other IBM courses.

Table of Contents

1. Exploring Datasets with *pandas*
2. The Dataset: Immigration to Canada from 1980 to 2013
3. *pandas* Basics
4. *pandas* Intermediate: Indexing and Selection
5. *pandas* Filtering based on a criteria
6. *pandas* Sorting Values

Exploring Datasets with *pandas*

pandas is an essential data analysis toolkit for Python. From their [website](#):

pandas is a Python package providing fast, flexible, and expressive data structures designed to make working with “relational” or “labeled” data both easy and intuitive. It aims to be the fundamental high-level building block for doing practical, **real world** data analysis in Python.

The course heavily relies on *pandas* for data wrangling, analysis, and visualization. We encourage you to spend some time and familiarize yourself with the *pandas* API Reference: <http://pandas.pydata.org/pandas-docs/stable/api.html>.

The Dataset: Immigration to Canada from 1980 to 2013

Dataset Source: [International migration flows to and from selected countries - The 2015 revision](#).

The dataset contains annual data on the flows of international immigrants as recorded by the countries of destination. The data presents both inflows and outflows according to the place of birth, citizenship or place of previous / next residence both for foreigners and nationals. The current version presents data pertaining to 45 countries.

In this lab, we will focus on the Canadian immigration data.

United Nations

Population Division

Department of Economic and Social Affairs

International Migration Flows to and from Selected Countries: The 2015 Revision

POPDB/MIG/Flow/Rev/2015

December 2015 - Copyright © 2015 by United Nations. All rights reserved

Suggested citation: United Nations, Department of Economic and Social Affairs, Population Division (2015). International Migration Flows to and from Selected Countries: The 2015 Revision and United Nations database, POPDB/MIG/Flow/Rev/2015.

Reporting country	Classification		Year																	
	Criteria	Type	1980	1981	1982	1983	1984	1985	1986	1987	1988	1989	1990	1991	1992	1993	1994	1995	1996	1997
Colombia	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Armenia	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Austria	Residence	Emigrants	96860	86600	82340	105610	90360	83440	63440	97710	104710	123040	121470	143710	143860	140420	141880	140880	159260	176960
	Residence	Immigrants	184290	212690	186200	153570	135330	122550	196690	221420	233860	238050	224500	227340	220460	197940	221810	253940	261330	260230
Australia	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Austria	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Austria	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Azerbaijan	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Belarus	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Belgium	Residence	Emigrants	13206	20305	21887	21090	20802	20681	21110	22250	18244	18076	19307	18002	13208	13616	14022	16642	16384	16200
	Residence	Immigrants	36887	36970	37207	36170	32747	30431	29520	31117	28661	24737	24373	31617	24597	29412	24042	31748	30616	32710
Belgium	Residence	Emigrants	794	7979	8479	8210	8643	9000	9663	9605	10200	10020	12160	13330	11173	10707	10182	9812	8638	8600
	Residence	Immigrants	38746	33007	29468	26877	26884	28009	28466	31160	31343	30584	29330	41763	43312	45344	51034	45014	47716	45067
Belgium	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Belgium	Residence	Emigrants	54694	48208	44659	43657	47002	47042	48959	49750	48484	54169	63862	67460	66763	63749	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Bulgaria	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Bulgaria	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Bulgaria	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Canada	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Canada	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Croatia	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Croatia	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Croatia	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Croatia	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Cyprus	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Cyprus	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Cyprus	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Cyprus	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Czech Republic	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Czech Republic	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Czech Republic	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Czech Republic	Residence	Emigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
	Residence	Immigrants	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..	..
Denmark	Residence	Emigrants	19778	18650	17991	16649	18880	17662	16668	19081	23880	26447	23328	22167	22057	22290	23391	23921	24305	24236
	Residence	Immigrants	11843	11077	10014	8722	8306	9171	8375	10068	10455	9273	86474	10185	9381	8814	8717	10881	10809	12889
Denmark	Residence	Emigrants	14626	14513	15035	15958	17742	18012	16349	16239	16605	19180	21000	21445	21883	20721	23884	20041	22018	24288
	Residence	Immigrants	12352	12982	12605	11433	12820	10219	20052	16217	16756	16666	17739	19744	16639	11623	20648	30238	28814	26853

The Canada Immigration dataset can be fetched from [here](#).

pandas Basics

The first thing we'll do is install **openpyxl** (formerly **xlrd**), a module that *pandas* requires to read Excel files.

```
In [ ]: !mamba install openpyxl==3.0.9 -y
```

Next, we'll do is import two key data analysis modules: *pandas* and *numpy*.

```
In [ ]: import numpy as np # useful for many scientific computing in Python
import pandas as pd # primary data structure library
```

Let's download and import our primary Canadian Immigration dataset using *pandas*'s `read_excel()` method.

```
In [ ]: df_can = pd.read_excel(
    'https://cf-courses-data.s3.us.cloud-object-storage.appdomain.cloud/IBMDevelope
    sheet_name='Canada by Citizenship',
    skiprows=range(20),
    skipfooter=2)

print('Data read into a pandas dataframe!')
```

Let's view the top 5 rows of the dataset using the `head()` function.

```
In [ ]: df_can.head()
# tip: You can specify the number of rows you'd like to see as follows: df_can.head
```

We can also view the bottom 5 rows of the dataset using the `tail()` function.

```
In [ ]: df_can.tail()
```

When analyzing a dataset, it's always a good idea to start by getting basic information about your dataframe. We can do this by using the `info()` method.

This method can be used to get a short summary of the dataframe.

```
In [ ]: df_can.info(verbose=False)
```

To get the list of column headers we can call upon the data frame's `columns` instance variable.

```
In [ ]: df_can.columns
```

Similarly, to get the list of indices we use the `.index` instance variables.

```
In [ ]: df_can.index
```

Note: The default type of instance variables `index` and `columns` are **NOT** `list`.

```
In [ ]: print(type(df_can.columns))
        print(type(df_can.index))
```

To get the index and columns as lists, we can use the `tolist()` method.

```
In [ ]: df_can.columns.tolist()
```

```
In [ ]: df_can.index.tolist()
```

```
In [ ]: print(type(df_can.columns.tolist()))
        print(type(df_can.index.tolist()))
```

To view the dimensions of the dataframe, we use the `shape` instance variable of it.

```
In [ ]: # size of dataframe (rows, columns)
        df_can.shape
```

Note: The main types stored in *pandas* objects are `float`, `int`, `bool`, `datetime64[ns]`, `datetime64[ns, tz]`, `timedelta[ns]`, `category`, and `object` (string). In addition, these dtypes have item sizes, e.g. `int64` and `int32`.

Let's clean the data set to remove a few unnecessary columns. We can use *pandas* `drop()` method as follows:

```
In [ ]: # in pandas axis=0 represents rows (default) and axis=1 represents columns.
        df_can.drop(['AREA', 'REG', 'DEV', 'Type', 'Coverage'], axis=1, inplace=True)
        df_can.head(2)
```

Let's rename the columns so that they make sense. We can use `rename()` method by passing in a dictionary of old and new names as follows:

```
In [ ]: df_can.rename(columns={'OdName': 'Country', 'AreaName': 'Continent', 'RegName': 'Region'})
        df_can.columns
```

We will also add a 'Total' column that sums up the total immigrants by country over the entire period 1980 - 2013, as follows:

```
In [ ]: df_can['Total'] = df_can.sum(axis=1)
        df_can['Total']
```

We can check to see how many null objects we have in the dataset as follows:

```
In [ ]: df_can.isnull().sum()
```

Finally, let's view a quick summary of each column in our dataframe using the `describe()` method.

```
In [ ]: df_can.describe()
```

pandas Intermediate: Indexing and Selection (slicing)

Select Column

There are two ways to filter on a column name:

Method 1: Quick and easy, but only works if the column name does NOT have spaces or special characters.

```
df.column_name          # returns series
```

Method 2: More robust, and can filter on multiple columns.

```
df['column']            # returns series  
df[['column 1', 'column 2']] # returns dataframe
```

Example: Let's try filtering on the list of countries ('Country').

```
In [ ]: df_can.Country # returns a series
```

Let's try filtering on the list of countries ('Country') and the data for years: 1980 - 1985.

```
In [ ]: df_can[['Country', 1980, 1981, 1982, 1983, 1984, 1985]] # returns a dataframe  
# notice that 'Country' is string, and the years are integers.  
# for the sake of consistency, we will convert all column names to string later on.
```

Select Row

There are main 2 ways to select rows:

```
df.loc[label]          # filters by the labels of the index/column  
df.iloc[index]         # filters by the positions of the index/column
```

Before we proceed, notice that the default index of the dataset is a numeric range from 0 to 194. This makes it very difficult to do a query by a specific country. For example to search for data on Japan, we need to know the corresponding index value.

This can be fixed very easily by setting the 'Country' column as the index using

`set_index()` method.

```
In [ ]: df_can.set_index('Country', inplace=True)  
# tip: The opposite of set is reset. So to reset the index, we can use df_can.reset
```

```
In [ ]: df_can.head(3)
```

```
In [ ]: # optional: to remove the name of the index
df_can.index.name = None
```

Example: Let's view the number of immigrants from Japan (row 87) for the following scenarios: 1. The full row data (all columns) 2. For year 2013 3. For years 1980 to 1985

```
In [ ]: # 1. the full row data (all columns)
df_can.loc['Japan']
```

```
In [ ]: # alternate methods
df_can.iloc[87]
```

```
In [ ]: df_can[df_can.index == 'Japan']
```

```
In [ ]: # 2. for year 2013
df_can.loc['Japan', 2013]
```

```
In [ ]: # alternate method
# year 2013 is the last column, with a positional index of 36
df_can.iloc[87, 36]
```

```
In [ ]: # 3. for years 1980 to 1985
df_can.loc['Japan', [1980, 1981, 1982, 1983, 1984, 1985]]
```

```
In [ ]: # Alternative Method
df_can.iloc[87, [3, 4, 5, 6, 7, 8]]
```

Exercise: Let's view the number of immigrants from **Haiti** for the following scenarios:

1. The full row data (all columns)
2. For year 2000
3. For years 1990 to 1995

```
In [ ]: df_can.loc['Haiti']
df_can.loc['Haiti', 2000]
df_can.loc['Haiti', [1990, 1991, 1992, 1993, 1994, 1995]]
```

► [Click here for a sample python solution](#)

Column names that are integers (such as the years) might introduce some confusion. For example, when we are referencing the year 2013, one might confuse that when the 2013th positional index.

To avoid this ambiguity, let's convert the column names into strings: '1980' to '2013'.

```
In [ ]: df_can.columns = list(map(str, df_can.columns))
# [print (type(x)) for x in df_can.columns.values] #<-- uncomment to check type of
```

Since we converted the years to string, let's declare a variable that will allow us to easily call upon the full range of years:

```
In [ ]: # useful for plotting later on
years = list(map(str, range(1980, 2014)))
years
```

Exercise: Create a list named 'year' using map function for years ranging from 1990 to 2013. Then extract the data series from the dataframe df_can for Haiti using year list.

```
In [ ]:
```

► [Click here for a sample python solution](#)

Filtering based on a criteria

To filter the dataframe based on a condition, we simply pass the condition as a boolean vector.

For example, Let's filter the dataframe to show the data on Asian countries (AreaName = Asia).

```
In [ ]: # 1. create the condition boolean series
condition = df_can['Continent'] == 'Asia'
print(condition)
```

```
In [ ]: # 2. pass this condition into the DataFrame
df_can[condition]
```

```
In [ ]: # we can pass multiple criteria in the same line.
# Let's filter for AreaName = Asia and RegName = Southern Asia

df_can[(df_can['Continent']=='Asia') & (df_can['Region']=='Southern Asia')]

# note: When using 'and' and 'or' operators, pandas requires we use '&' and '|' ins
# don't forget to enclose the two conditions in parentheses
```

Exercise: Fetch the data where AreaName is 'Africa' and RegName is 'Southern Africa'. Display the dataframe and find out how many instances are there?

```
In [ ]:
```

► [Click here for a sample python solution](#)

Sorting Values of a Dataframe or Series

You can use the `sort_values()` function is used to sort a DataFrame or a Series based on one or more columns.

You to specify the column(s) by which you want to sort and the order (ascending or descending). Below is the syntax to use it:-

```
df.sort_values(col_name, axis=0, ascending=True, inplace=False,
ignore_index=False)
```

col_name - the column(s) to sort by.

axis - axis along which to sort. 0 for sorting by rows (default) and 1 for sorting by columns.

ascending - to sort in ascending order (True, default) or descending order (False).

inplace - to perform the sorting operation in-place (True) or return a sorted copy (False, default).

ignore_index - to reset the index after sorting (True) or keep the original index values (False, default).

Let's sort out dataframe df_can on 'Total' column, in descending order to find out the top 5 countries that contributed the most to immigration to Canada.

```
In [ ]: df_can.sort_values(by='Total', ascending=False, axis=0, inplace=True)
top_5 = df_can.head(5)
top_5
```

Exercise: Find out top 3 countries that contributes the most to immigration to Canada in the year 2010.

Display the country names with the immigrant count in this year

```
In [ ]:
```

► [Click here for a sample python solution](#)

Congratulations! you have learned how to wrangle data with Pandas. You will be using alot of these commands to preprocess the data before its can be used for data visualization.

Thank you for completing this lab!

Author

[Alex Akson](#)

Other Contributors

[Jay Rajasekharan](#), [Ehsan M. Kermani](#), [Slobodan Markovic](#), [Weiqing Wang](#), [Dr. Pooja](#)

<!-- --!>

Change Log

| Date (YYYY-MM-DD) | Version | Changed By | Change Description |
|-------------------|---------|---------------|---|
| 2023-06-08 | 2.5 | Dr. Pooja | Separated from original lab |
| 2021-05-29 | 2.4 | Weiying Wang | Fixed typos and code smells. |
| 2021-01-20 | 2.3 | Lakshmi Holla | Changed TOC cell markdown |
| 2020-11-20 | 2.2 | Lakshmi Holla | Changed IBM box URL |
| 2020-11-03 | 2.1 | Lakshmi Holla | Changed URL and info method |
| 2020-08-27 | 2.0 | Lavanya | Moved Lab to course repo in GitLab --!> |

© IBM Corporation 2020. All rights reserved.

In []: